

Thinking mode and a 32B policy on CodeGym: two additions to the SUPO Table-1 replication

Xiaoxuan Lei

ByteDance / McGill — runs 2026-09-08 to 2026-09-10 (PDT), ark-eng-algorithm H100 queue

September 10, 2026

Abstract

We extend the SUPO Table-1 replication on CodeGym (report of 2026-09-09: Qwen3.5-9B, GRPO-32K vs. SUPO-4K×8) with two questions. (A) Does letting the policy *think* (Qwen3.5 thinking mode, reasoning tokens packed into the training context and trained) change the outcome of either recipe? (B) How does the model used in the papers, Qwen2.5-32B-Instruct, behave under the same GRPO-32K recipe on the same 128 held-out tasks? All five arms were trained for 100 steps and evaluated greedily every 10 steps on the same 128 tasks. Thinking does not help: GRPO-32K reaches 0.859 with thinking vs. 0.867 without, and SUPO-4K×8 reaches 0.695 vs. 0.758, while GRPO with thinking spends most of the run with 15–26% of episodes truncated at the 32K cap. Qwen2.5-32B-Instruct starts far lower (0.523 vs. 0.766 for Qwen3.5-9B) and gains three times as much from RL (+0.312 vs. +0.102), finishing at 0.836; the two final policies solve 95 of the 128 tasks in common and each solves 19 of the 30 tasks the 9B base model failed. Our 32B base accuracy (0.52) is not comparable with the 30.1% in-domain figure of the CodeGym paper, whose evaluation set is filtered to tasks the base model fails. The 2-node training and checkpoint-mirror pipeline needed sixteen smoke runs to reach a clean state; every fix is listed, and the final run mirrored its checkpoints from both nodes at 400–460 MB/s.

1 Introduction

The base replication (`docs/report/REPORT.md`) trained Qwen3.5-9B on 12,800 CodeGym tasks with two context-management recipes from SUPO [1]: vanilla multi-turn GRPO with a 32K working context, and SUPO with a 4K working context and up to seven summaries. Both were run in the model’s non-thinking mode. Two follow-up questions were raised: whether the policy’s chain-of-thought (thinking mode) should be part of the trained context, and how the papers’ own model, Qwen2.5-32B-Instruct [2], compares on our evaluation set. This report answers both with the same data pipeline, evaluation set and step budget as the base replication, so all five arms are directly comparable on the same 128 tasks.

2 Setup

Everything not listed in Table 1 is identical to §2 of the base report: 12,800 training tasks (at most two per seed problem, oracle solution ≤ 80 calls), 128 held-out tasks from 128 unseen seeds whose oracle needs 40–90 calls, 100 steps of 128 prompts $\times$ 8 rollouts, lr 10^{-6} , clip 0.20/0.28, token-mean loss, no KL or entropy bonus, 100 LLM calls per episode, greedy pass@1 validation every 10 steps, reward from the CodeGym Done verifier.

Table 1: Arm-specific configuration. “Packed” means the thinking tokens are kept in the trajectory token stream that the policy is trained on (verl’s continuous-token builder appends each generated turn, thinking included), not only shown to the model at generation time.

arm	configuration
9B GRPO-32K, 9B SUPO-4K×8	base-report arms (run 5): Qwen3.5-9B, enable_thinking=false (chat template emits an empty <think> block), 1×8 H100 each.
9B GRPO-32K+think	enable_thinking=true; per-turn cap 4,096 tokens (was 1,024); thinking packed and trained; working context 32,768 (prompt 2,048 / response 30,720); 1×8 H100.
9B SUPO-4K×8+think	as above plus summary-turn cap 2,048 tokens (was 1,024); per-trajectory response 8,192 and max_model_len 12,288 (was 5,120 / 10,240) so that a 4K working context plus a 4K thinking turn fits; ≤ 7 summaries; 1×8 H100.
32B GRPO-32K	Qwen2.5-32B-Instruct (its native template has no thinking), 2×8 H100, FSDP2 actor with tensor-parallel 4 vLLM rollouts and Ulysses sequence-parallel 4, fused log-prob kernels, optimizer offload; max_model_len 32,768 = 2,048 + 30,720 (the model’s position limit, so the response budget is 30,720 as for the 9B arm); checkpoints every 20 steps mirrored per node to HDFS; weights fetched per file from Hugging Face at pod start.

Infrastructure. Each arm is one Merlin/Arnold batch job (group ark-eng-algorithm) whose pods restore a pinned venv (torch 2.11+cu129, vLLM 0.24, flash-attn 2.8.3) and a repo snapshot from HDFS, stage the model to local disk, start a ray cluster (rank 0 head, rank 1 joins over the published IPv4 address), train with scripts/train_codegym.sh, and mirror each local checkpoint to HDFS with scripts/merlin/ckpt_sync.sh. Metrics go to the Merlin tracking project supo_codegym (byted-wandb overlay; run name = experiment name), see https://ml.tiktok-row.net/experiment/tracking/detail?Id=project_20260907_b5e8830b. All figures and tables below are produced by scripts/make_report_thinking_32b.py from the validation dumps, the trainer logs and the sampled rollout dumps on HDFS; assets/README.md documents every data file.

Deviations from the papers and from the base report (honest list).

- The evaluation set is ours (128 tasks, oracle 40–90 calls, no accuracy filter, greedy decoding, 100-call budget), not the CodeGym paper’s (Section 3.5). Standard error at $p \approx 0.8$ is ± 0.035 ; differences below ~ 0.05 between arms are within noise.
- Thinking arms raise the per-turn token cap from 1,024 to 4,096 (and SUPO’s per-trajectory response from 5,120 to 8,192); the working-context sizes (32K / 4K) are unchanged. Thinking therefore competes with tool calls for the same context budget, which is the point of the comparison but also means the thinking arms are not a pure “add reasoning” ablation.
- Qwen2.5-32B-Instruct’s template has no thinking switch; its arm is non-thinking by construction.
- The 32B arm trains on 16 GPUs with TP4/SP4 and fused kernels; the batch and optimizer are the same as the 9B arm, so per-step samples are identical, but the numerics (bf16 hidden states cast to the fp32 lm-head dtype inside the fused cross-entropy kernel) differ slightly from the 9B path.
- Training-log curves for the two non-thinking 9B arms start at step 36 / 40 because those runs were resumed from HDFS checkpoints and only the resumed pod’s log survived; their validation points from step 0 are complete. The 9B GRPO arm also has a validation point at step 35 from the pre-resume pod.
- The environment Codeforces_5537_I__MinPathCostEnv fails to start under Python 3.12 (f-string syntax); its episodes are replaced by a zero-reward fallback row in all arms.
- No wandb sync was possible: the byted-wandb overlay logs to Merlin tracking, not to a local wandb store, so batch-level curves are parsed from the trainer stdout instead.

3 Results

3.1 Validation accuracy

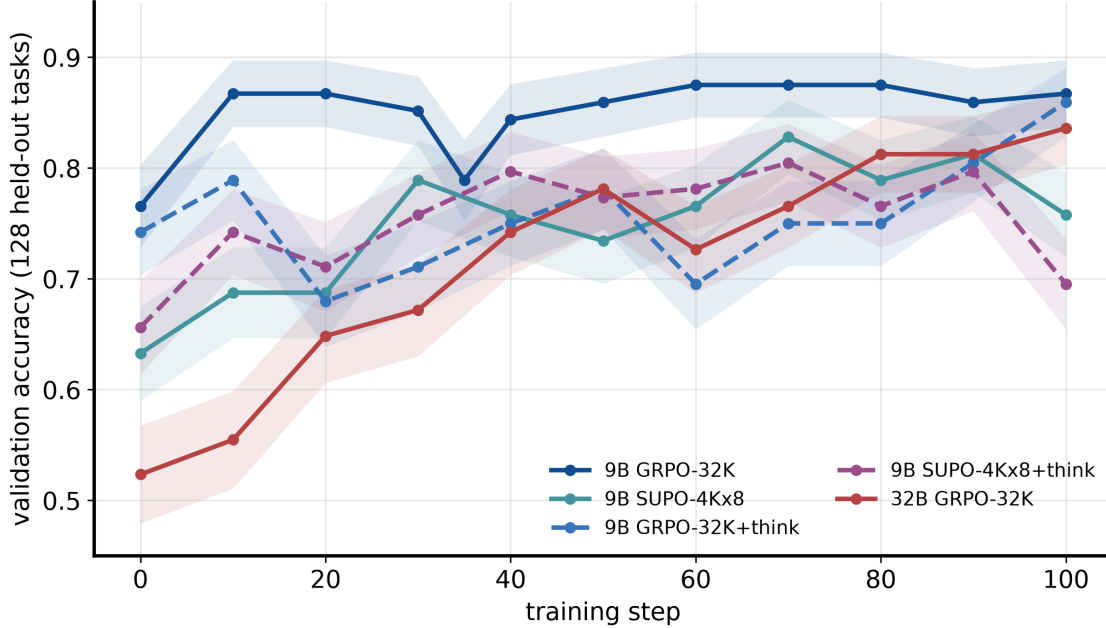


Figure 1: Greedy validation accuracy on the 128 held-out CodeGym tasks after every 10 training steps, for the two base-report arms (solid, blue/teal), their thinking variants (dashed) and the Qwen2.5-32B-Instruct GRPO arm (red). Shaded bands are ± 1 standard error $\sqrt{p(1-p)/128}$. Each point is the mean of the binary score field over the 128 records of outputs/<exp>/val/<step>.jsonl. Thinking hurts GRPO for most of the run and only catches up at step 100; thinking never helps SUPO; the 32B model starts 0.24 below the 9B model and closes most of the gap by step 80. Data: fig1_val_acc_all_arms.csv.

Table 2: Validation accuracy per checkpoint (greedy, $n = 128$). “final” is step 100, “peak” the best checkpoint, “mean 80–100” the mean of the last three checkpoints. Source: tables.md / val_metrics.csv.

arm	0	10	20	30	40	50	60	70	80	90	100	final	peak	80–100
9B GRPO-32K	.766	.867	.867	.852	.844	.859	.875	.875	.875	.859	.867	.867	.875 @60	.867
9B SUPO-4Kx8	.633	.688	.688	.789	.758	.734	.766	.828	.789	.812	.758	.758	.828 @70	.786
9B GRPO-32K+think	.742	.789	.680	.711	.750	.781	.695	.750	.750	.805	.859	.859	.859 @100	.805
9B SUPO-4Kx8+think	.656	.742	.711	.758	.797	.773	.781	.805	.766	.797	.695	.695	.805 @70	.753
32B GRPO-32K	.523	.555	.648	.672	.742	.781	.727	.766	.812	.812	.836	.836	.836 @100	.820

Thinking verdict. On the same tasks and step budget, enabling thinking changes the final accuracy by -0.008 for GRPO-32K ($0.867 \rightarrow 0.859$; within noise) and by -0.063 for SUPO-4Kx8 ($0.758 \rightarrow 0.695$; peaks 0.828 vs. 0.805). Averaged over the last three checkpoints, which is the more robust summary, thinking costs 0.062 for GRPO ($0.867 \rightarrow 0.805$) and 0.033 for SUPO ($0.786 \rightarrow 0.753$). The thinking arms also cost more compute: median step time 946 s vs. 794 s for GRPO (Table 3).

32B vs. 9B. Qwen2.5-32B-Instruct starts at 0.523 and ends at 0.836 ($+0.312$), Qwen3.5-9B at 0.766 and 0.867 ($+0.102$). The 32B curve is still rising at step 100 ($0.812, 0.812, 0.836$ over the last three checkpoints)

whereas the 9B curve is flat from step 10 on.

3.2 Behaviour: truncation, tool calls, thinking length

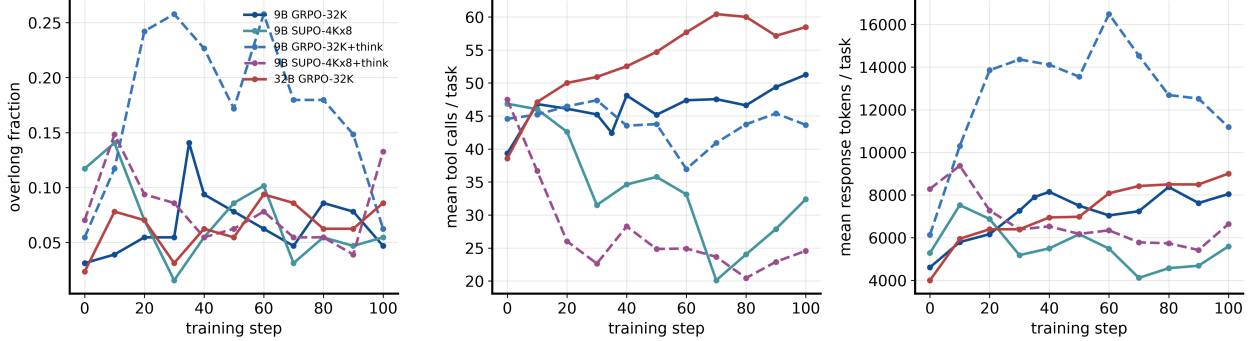


Figure 2: Per-checkpoint behaviour on the validation set (mean over the 128 episodes). Left: fraction of episodes flagged overlong (the episode hit the 100-call limit, the working-context limit, or ten consecutive turns without a function call). Middle: mean number of tool calls per episode. Right: mean generated tokens per episode (rollout_response_tokens, thinking included). GRPO+think truncates 15–26% of episodes between steps 20 and 90 while generating ~11K tokens per episode; the non-thinking GRPO arm stays at 3–9% and ~8K. SUPO+think keeps truncation low (4–9%) except at step 100 (13%) and makes the fewest tool calls (24.6 at step 100). Data: fig2_overlong_calls_tokens.csv.

Table 3: Behaviour at the first and last checkpoint and training cost. Overlong and tool calls from the validation dumps; the median step time is timing_s/step over the logged steps of each arm’s trainer log (see the deviations list for the two resumed logs). Source: summary.json.

arm	overlong @0	overlong @100	tool calls @0	tool calls @100	summaries @100	median step (s)
9B GRPO-32K	0.03	0.05	39.3	51.2	0.00	794
9B SUPO-4K×8	0.12	0.05	46.9	32.4	1.32	539
9B GRPO-32K+think	0.05	0.06	44.6	43.6	0.00	946
9B SUPO-4K×8+think	0.07	0.13	47.5	24.6	1.98	551
32B GRPO-32K	0.02	0.09	38.6	58.5	0.00	639 (16 GPUs)

The mechanism behind the GRPO+think dip is visible across Figures 2–3: the policy’s thinking grows from step 10 on, the per-episode generation grows to ~11K tokens, more episodes overrun the 32K window and are masked out of the loss, and validation accuracy falls to 0.68–0.75. Between steps 60 and 100 the mean think length shrinks again (361 → 191 tokens per thinking turn), truncation falls from 26% to 6%, and accuracy recovers to 0.859. SUPO absorbs the extra thinking tokens through summarization, so its truncation stays low, but its thinking policy ends with fewer tool calls (24.6 vs. 32.4) and a lower final score.

3.3 Training curves

3.4 32B vs. 9B on the same tasks

3.5 Comparison with the CodeGym paper

The CodeGym paper [2] reports, for Qwen2.5-32B-Instruct, an in-domain success rate of 30.1% before RL and 81.0% after (75.0% without its data filters). Table 4 lists why those levels are not comparable with ours, although the direction and the size of the RL gain are. The paper’s $\leq 25\%$ accuracy screen fixes the

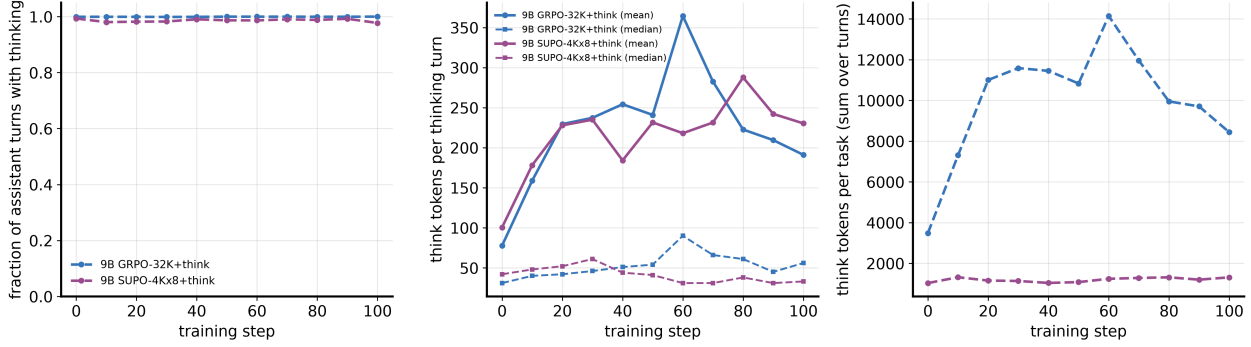


Figure 3: Thinking statistics of the two thinking arms per checkpoint, measured on the greedy validation outputs (output field of the validation dumps: the decoded generation including the chat-template role text). Each assistant turn is obtained by splitting on the template’s assistant marker; a turn counts as a thinking turn when the text before its `</think>` tag (opening tag removed; for the first turn the template supplies the opening tag inside the prompt) is non-empty, or when a `<think>` block is opened but never closed (the turn was cut by the 4,096-token per-turn cap). Think tokens are counted with the Qwen3.5-9B tokenizer. Left: fraction of turns with thinking (99–100% for GRPO+think, 96–99% for SUPO+think; the non-thinking control arms measure exactly 0). Middle: mean (solid) and median (dashed) think tokens per thinking turn; the median stays at 30–60 tokens while the mean grows from 78 (step 0) to 240–360 (steps 40–60) and back to 190 (step 100) for GRPO+think, i.e. a heavy tail of very long thoughts (p90 up to 610 tokens, maximum 4,140 = the cap). Right: total think tokens per validation episode; GRPO+think spends 3.5K tokens per episode on thinking at step 0 and 8–14K in the middle of the run. Limitation: for SUPO the output field holds only the final working-context segment of each episode (600–1,300 turns per 128 tasks instead of ~5,000), so its per-episode totals cover that segment only. Data: `fig3_thinking_stats.csv`.

base score near the screening threshold by construction, and its 50-point gain is measured on tasks selected because the base model fails them. Our set never removes tasks the base model solves, so a strong model starts high; the 32B base at 0.52 (0.39–0.50 in earlier two-step smoke runs, $n = 128$ each) is the same model on a less adversarial pool with greedy decoding. A direct number against the paper would require rebuilding an evaluation split with both of its filters and re-scoring the saved checkpoints (steps 80 and 100 of the 32B run and steps 95/100 of the 9B runs are on HDFS); that is a data-preparation change, not a retrain.

4 Example trajectories

Excerpts rendered by `pretty_traj` in the builder script (tool outputs truncated, long runs of turns elided); the full excerpts are in `assets/traj_*.txt` and the records in `trajectories.json`.

32B, step 0, failure (sampled training rollout). The base model follows the protocol but merges the strings one character at a time and submits a wrong answer after 62 calls.

```
env: Leetcode_31007_I__StringMergeEnv
config: {"word1": "abcdefghijklmnopqrstuvwxyz", "word2": "123"}
reward=0.0 finished=True overlong=False stop=done tool_calls=62 summaries=0 steps=62
--- trajectory 1/1 (prompt 849 tok, response 7864 tok)
task: Please answer the following question step by step according to the requirements below! 1. **
Do not** write code to answer the user's question -you may only call the provided functions,
and you may call at most **one function per step**. 2. After you call a function, wait for the
tool to return the result -do not assume what the result will be. 3. If the 'tools
description is unclear, you can try using it first, and then adjust your function call based
```

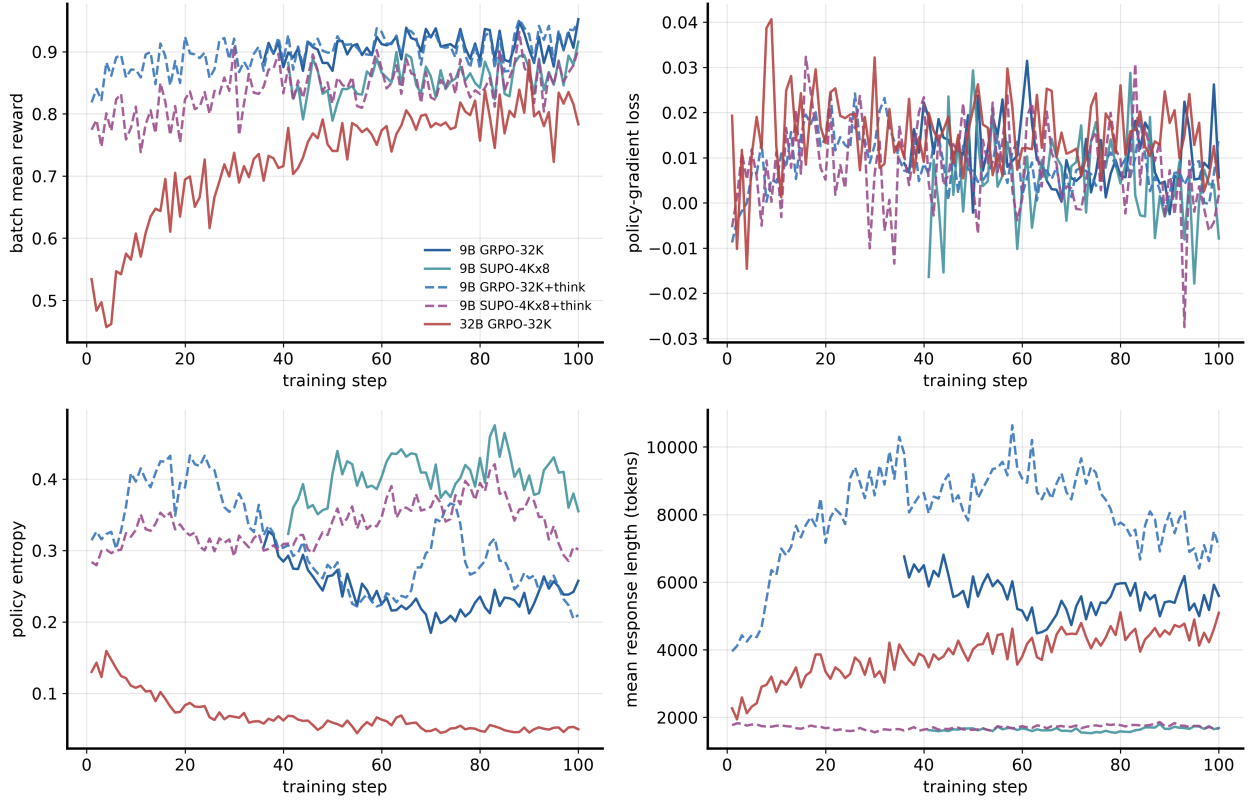
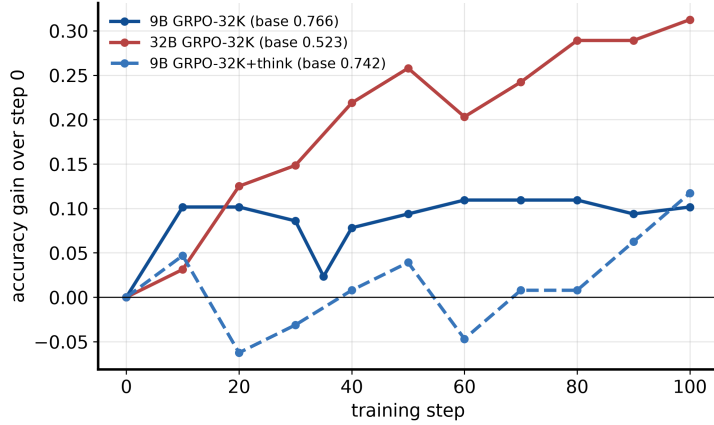


Figure 4: Batch-level trainer metrics parsed from the step:N – key:value lines of each arm’s trainer stdout (ray and ANSI prefixes stripped, last occurrence per step kept). Top left: mean training reward of the batch (critic/score/mean, 128 prompts $\times$ 8 rollouts at temperature 1). Top right: policy-gradient loss. Bottom left: policy entropy (actor/entropy). Bottom right: mean generated tokens per rollout (response_length/mean). The 9B non-thinking curves start at step 36/40 (resumed pods, see deviations). The 32B policy has a much lower entropy from the start (0.13 $\rightarrow$ 0.05) than the 9B policies (0.26–0.43) and its response length grows steadily (2.3K $\rightarrow$ 5.1K tokens) as it learns to make more calls per episode (38.6 $\rightarrow$ 58.5 on validation). GRPO+think produces the longest rollouts (up to 10.6K tokens at step 58) and its entropy rises during the dip and falls again as thinking shortens. Data: fig4_training_curves.csv.

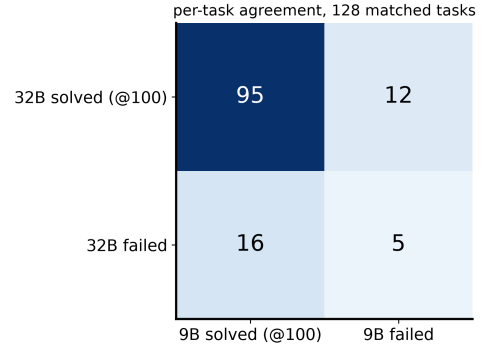
```

on the returned result. 4. Function calls should be wrapped with `<|FunctionCallBegin|>...<|
FunctionCallEnd|>` and contain a JSON-formatted l
A1: To begin with, let's observe the current environment state to understand what strings we are
dealing with. <|FunctionCallBegin|>[{"name": "Observe", "parameters": {}}]<|FunctionCallEnd|>
obs: Current environment: word1 and word2 are ready, merged list not created
A2: The environment indicates that the strings `word1` and `word2` are ready, but the merged list
has not been created yet. Let's create the merged list now. <|FunctionCallBegin|>[{"name": "
CreateMergedList", "parameters": {}}]<|FunctionCallEnd|>
obs: New merged list has been created
A3: The merged list has been successfully created. Next, we need to get the lengths of `word1`
and `word2` to determine how many characters we need to process. <|FunctionCallBegin|>[{"name
": "GetStringLength", "parameters": {"string_name": "word1"}}]<|FunctionCallEnd|>
obs: 26
A4: The length of `word1` is 26. Now, let's get the length of `word2`. <|FunctionCallBegin|>[{"
name": "GetStringLength", "parameters": {"string_name": "word2"}}]<|FunctionCallEnd|>
obs: 3
A5: The length of `word2` is 3. Now we know the lengths of both strings. We will start merging
the strings by taking characters alternately from `word1` and `word2`. Let's start with the

```



(a) Accuracy gain over the arm's own step-0 accuracy.



(b) Per-task agreement at step 100.

Figure 5: (a) Gain over base for the three GRPO-32K arms: the 32B model gains +0.31 against +0.10 for the 9B model, but from a base 0.24 lower. (b) Solved/unsolved contingency between the final 32B and 9B GRPO policies on the 128 tasks, matched across runs by an MD5 of the validation prompt text with chat-template role markers, thinking scaffolds and whitespace removed (128 of 128 matched). Both solve 95 tasks; 16 are solved only by the 9B policy, 12 only by the 32B policy, 5 by neither. Of the 30 tasks the 9B *base* model failed at step 0, the final 9B policy solves 19 and the final 32B policy also solves 19. Data: fig5_gain_over_base.csv, fig6_task_agreement.json.

Table 4: Evaluation protocol of the CodeGym paper (from its text) vs. this report.

	CodeGym paper (arXiv 2509.17325)	this report
evaluation set	500 unseen environments, 972 task configurations	128 unseen seed problems, 128 tasks
difficulty filter	10–256 required tool calls <i>and</i> base-model accuracy $\leq 25\%$	oracle solution 40–90 calls; no accuracy filter
decoding	temperature 0.7, top-p 0.95, pass@1	greedy, one attempt
tool-call budget	256 (512 for the hard variant)	100
context	5,120 prompt + 24,576 response	2,048 + 30,720
training	GRPO, 512 prompts $\times$ 8, $\sim$ 100 steps	GRPO, 128 prompts $\times$ 8, 100 steps
Qwen2.5-32B-Instruct	30.1 $\rightarrow$ 81.0 (filtered data) / 75.0 (unfiltered)	0.523 $\rightarrow$ 0.836

```

first character of `word1`. <|FunctionCallBegin|>[{"name": "GetCharacterAtPosition", "
parameters": {"string_name": "word1", "position": 0}}]<|FunctionCallEnd|>
obs: a
A6: The first character of `word1` is 'a'. Now, let's ad
... [elided] ...
', '2']. Next, we will get the third character of `word1` and add it to the merged list. <|
FunctionCallBegin|>[{"name": "GetCharacterAtPosition", "parameters": {"string_name": "word1",
"position": 2}}]<|FunctionCallEnd|>
obs: c
A14: The third character of `word1` is 'c'. Now, let's add this character to the merged list. <|
FunctionCallBegin|>[{"name": "AddCharacterToList", "parameters": {"character": "c"}}]<|
FunctionCallEnd|>
obs: Character 'c' has been added to the merged list, current list: ['a', '1', 'b', '2', 'c']
... (assistant turns elided) ...
A62: The merged string is "alb2cdefghijklmnopqrstuvwxyz". Now, let's verify whether this is the
correct final answer. <|FunctionCallBegin|>[{"name": "Done", "parameters": {"answer": "
alb2cdefghijklmnopqrstuvwxyz"}}]<|FunctionCallEnd|>

```

32B, step 100, success. Same environment as the failure below (two rollouts of the same prompt in the step-100 batch). The policy recovers from a malformed first call, plans the full search, and submits the right answer after 79 calls.

```
env: Codeforces_38332_I_MinLandMassesEnv
config: {"n": 5, "m": 7, "grid": [
  ["L", "W", "L", "L", "L", "W", "L"],
  ["L", "L", "L", "W", "L", "L", "L"],
  ["W", "L", "W", "L", "W", "L", "W"],
  ["L", "L", "L", "L", "L", "L", "L"],
  ["L", "L", "L", "L", "L", "L", "L"]
]}
reward=1.0 finished=True overlong=False stop=done tool_calls=79 summaries=0 steps=80
--- trajectory 1/1 (prompt 687 tok, response 9450 tok)
task: Please answer the following question step by step according to the requirements below! 1. **Do not** write code to answer the user's question -you may only call the provided functions, and you may call at most one function per step**. 2. After you call a function, wait for the tool to return the result -do not assume what the result will be. 3. If the 'tools' description is unclear, you can try using it first, and then adjust your function call based on the returned result. 4. Function calls should be wrapped with `<|FunctionCallBegin|>...<|FunctionCallEnd|>` and contain a JSON-formatted 1
A1: To solve this problem, we need to follow these steps: 1. Observe the Initial Grid: Use the `Observe()` function to get the current state of the grid. 2. Count Initial Landmasses: Use the `CountLandMasses()` function to find the initial number of landmasses in the grid. 3. Iterate Over Each Land Cell: For each land cell ('L') in the grid: - Temporarily change the cell to water ('W') using the `ChangeLandTo`
obs: Error: The action cannot be parsed in json format: Unterminated string starting at: line 1 column 37 (char 36)
A2: It appears there is an error due to an incorrect format in the function call. Let's correct the format and call the `Observe()` function again to get the current state of the grid. <|FunctionCallBegin|>[{"name": "Observe", "parameters": {}}]<|FunctionCallEnd|>
obs: [
  ["L", "W", "L", "L", "L", "W", "L"],
  ["L", "L", "L", "W", "L", "L", "L"],
  ["W", "L", "W", "L", "L", "L", "L"],
  ["L", "L", "L", "L", "L", "L", "L"],
  ["L", "L", "L", "L", "L", "L", "L"]
]
A3: The current state of the grid is as follows: `` L W L L L W L L L W L L L W L W L W L W L L L W L L L W L L L W L `` Next, we need to count the initial number of landmasses in this grid. Let's use the `CountLandMasses()` function for this purpose. <|FunctionCallBegin|>[{"name": "CountLandMasses", "parameters": {}}]<|FunctionCallEnd|>
... [elided] ...
, 4). We will change the land cell at `(0, 4)` to water, count the number of landmasses, and then reset the grid. Let's start by changing the land cell at `(0, 4)` to water. <|FunctionCallBegin|>[{"name": "ChangeLandToWater", "parameters": {"row": 0, "col": 4}}]<|FunctionCallEnd|>
obs: Successfully changed the land at (0,4) to water
... (assistant turns elided) ...
A80: The grid has been successfully reset to its initial state. We have now iterated through all the land cells and recorded the number of landmasses after changing each one to water. The minimum number of landmasses found after changing one land cell to water is 1. This minimum was achieved when we changed the land cell at `(2, 3)` to water. Now, we will verify the final answer using the `Done()` function. <|Function
```

32B, step 100, failure. The sibling rollout re-observes the grid after every reset, runs out of the 100-call budget (stop=max_steps, overlong) and never submits.

```
env: Codeforces_38332_I_MinLandMassesEnv
config: {"n": 5, "m": 7, "grid": [
  ["L", "W", "L", "L", "L", "W", "L"],
  ["L", "L", "L", "W", "L", "L", "L"],
  ["W", "L", "W", "L", "W", "L", "W"],
  ["L", "L", "L", "L", "L", "L", "L"],
  ["L", "L", "L", "L", "L", "L", "L"]
]}
reward=0.0 finished=False overlong=True stop=max_steps tool_calls=99 summaries=0 steps=100
--- trajectory 1/1 (prompt 687 tok, response 16102 tok)
```

task: Please answer the following question step by step according to the requirements below! 1. ****Do not**** write code to answer the user's question -you may only call the provided functions, and you may call at most ****one** function per step******. 2. After you call a function, wait for the tool to return the result -do not assume what the result will be. 3. If the 'tools description is unclear, you can try using it first, and then adjust your function call based on the returned result. 4. Function calls should be wrapped with `<|FunctionCallBegin|>...<|FunctionCallEnd|>` and contain a JSON-formatted 1

A1: To solve this problem, we need to follow these steps: 1. ****Observe the initial grid state****: We need to know the current configuration of the grid. 2. ****Count the initial number of landmasses****: This gives us a baseline to compare future changes. 3. ****Iterate over each land cell in the grid****: For each land cell, we will: - Change the land cell to water. - Count the number of landmasses after the change. - Record the

obs: Error: The action cannot be parsed in json format: Unterminated string starting at: line 1 column 37 (char 36)

A2: It appears there's an error due to an incorrect format in the function call. Let's correct that and try calling the `Observe` function again to get the initial grid state. `<|FunctionCallBegin|>[{"name": "Observe", "parameters": null}]<|FunctionCallEnd|>`

obs: `[["L", "W", "L", "L", "L", "W", "L"], ["L", "L", "L", "W", "L", "L", "L"], ["W", "L", "W", "L", "W", "L", "W"], ["L", "L", "L", "W", "L", "L", "L"], ["L", "W", "L", "L", "L", "W", "L"]]`

A3: The initial grid state is: ``` L W L L L W L L L W L L L W L W L W L W L L L W L L L W L ``` Next, we need to count the initial number of landmasses in this grid to establish a baseline. `<|FunctionCallBegin|>[{"name": "CountLandMasses", "parameters": null}]<|FunctionCallEnd|>`

obs: 2

A4: The initial number of landmasses in
... [elided] ...

nge to restore the original state of the grid. Let's revert the change by resetting the grid. `<|FunctionCallBegin|>[{"name": "ResetGrid", "parameters": null}]<|FunctionCallEnd|>`

obs: Grid has been reset to initial state

A14: The grid has been successfully reset to its initial state. Now, we will proceed to the next cell to check if changing it to water results in a smaller number of landmasses. The next cell to check is (0, 3). Let's first observe the current grid state to confirm: `<|FunctionCallBegin|>[{"name": "Observe", "parameters": null}]<|FunctionCallEnd|>`

obs: `[["L", "W", "L", "L", "L", "W", "L"], ["L", "L", "L", "W", "L", "L", "L"], ["W", "L", "W", "L", "W", "L", "W"], ["L", "L", "L", "W", "L", "L", "L"], ["L", "W", "L", "L", "L", "W", "L"]]`

... (assistant turns elided) ...

```
source: greedy validation output, grpo_codegym_qwen35-9b_32k_think @ step 100, first solved task (
  score=1)
  turn 1 <think> The user wants me to find the maximum rectangular area that can be formed by
    consecutive buildings, where each building has width 1, and the height of the rectangle is
    determined by the minimum height among the buildings in that rectangle. This is a classic
    problem known as "Largest Rectangle in Histogram" problem. The standard solution uses a stack
    -based approach: 1. We iterate through all buildings 2. We maintain a stack of indices where
    heights are in increasing order 3. When we encounter a b ... </think>
  turn 1 answer:
  turn 2 <think> I need to start solving the problem. Let me first understand what I need to do: 1.
    This is the "Largest Rectangle in Histogram" problem 2. I need to use a stack-based approach
    3. I need to call functions step by step Let me start by: 1. First, I should initialize the
    stack 2. Get the length of the building heights array 3. Then I can start processing Let me
    start: Step 1: Initialize stack Step 2: Get length of heights array Let me call
    InitializeStack() first. </think>
  turn 2 answer: <[FunctionCallBegin]>[{"name": "InitializeStack", "parameters": {}}]<[
```

```

FunctionCallEnd|>
obs: Stack has been initialized
turn 3 <think> Good! Stack is initialized. Now I need to get the length of the building heights
array. </think>
turn 3 answer: <|FunctionCallBegin|>[{"name": "GetLength", "parameters": {}}]<|FunctionCallEnd|>
obs: 7
turn 4 <think> Great! The length is 7. So we have 7 buildings. Now I need to: 1. Get the heights
of all buildings to understand the problem 2. Implement the stack-based algorithm Let me
first get all the heights to understand the problem better. I should get height at each index
from 0 to 6. Let me start by getting height at index 0: </think>
turn 4 answer: <|FunctionCallBegin|>[{"name": "GetHeight", "parameters":{"index": 0}}]<|
FunctionCallEnd|>
obs: 0

```

5 Incident log and timeline (PDT)

- **Sep 8, 16:03** thinking arms launched (1×8 H100 each); both ran uninterrupted to step 100 (SUPO+think done Sep 9 10:09, GRPO+think done Sep 9 20:31); both final checkpoints mirrored to HDFS.
- **Sep 8, 19:00–Sep 9, 13:00** sixteen 2-node smoke runs (2 steps each) for the 32B arm, each exposing one defect: huggingface_hub snapshot download hanging at ~31 GB (replaced by per-file curl); max_model_len above the model’s 32,768 positions; ray head start failing silently on the allocated port (port fallbacks, worker-port range pinned to 30000–39999); wandb “No API key” because ray daemons started before the env exports (train_env.sh sourced first); a verl bug in dense_common.forward_with_triton_backend passing the fp32 lm-head weight with bf16 hidden states to the fused cross-entropy kernel (cast added, mirrored from the Qwen3.5 path); on A100 pods an InfiniBand GPU-Direct registration error (NCCL_NET_GDR_LEVEL=LOC) and later a whole rack of IPv6-only pods (fail-fast preflight); a stale DONE marker in the shared run directory making the rank-1 pod leave the ray cluster at start (per-trial markers); verl’s driver actor landing on rank 1 so that rank 0 never saw the checkpoint tracker file (mirror readiness now derived from each node’s own shard files); HDFS-fuse dropping appends from a second writer (sync log made pod-local with a single-writer copy); and mawk printing the 196 GB byte sum in scientific notation, which aborted the upload in bash arithmetic (printf %.0f).
- **Sep 9, 10:47** the H100 smoke that had waited 13 h in the queue trained correctly on H100 (val 0.500/0.523/0.484 over two steps).
- **Sep 9, 13:27** the queued smoke was converted into the full run at the user’s request; pods after 92 min; trainer up at 15:06; 8–11 min per step.
- **Sep 9, 18:34** first live checkpoint (step 20) mirrored from both nodes in 450 s (433–438 MB/s over the fuse fallback after the HDFS CLI puts failed fast with an empty error); steps 40/60/80/100 likewise (399–463 MB/s).
- **Sep 10, 09:49** 32B run exited cleanly; checkpoints 80 and 100 complete on HDFS (two kept).

Open items. The HDFS CLI put path fails immediately on the 32B pods (empty error text; the fuse fallback carried every upload, so low priority). The IPv6-only preflight may be stricter than needed now that the stale-marker bug, which produced the “IPv6 hang”, is fixed. A paper-protocol evaluation split (call-count band plus base-accuracy screen) would enable a direct comparison with the CodeGym paper. Post-hoc evaluation of the saved checkpoints on that split is straightforward.

6 Discussion and limitations

- **Thinking is not free context.** With a fixed 32K window, a policy that thinks 3–14K tokens per episode leaves less room for tool calls and observations; GRPO’s reward signal then pushes it to think less (Fig. 3), and only when it does does accuracy recover. SUPO’s summaries hide the cost from the context but not from the policy: the thinking SUPO arm makes fewer calls and scores lower. On CodeGym, where the useful reasoning is a short plan followed by many cheap calls, thinking buys nothing at this scale.
- **Model comparison.** Qwen2.5-32B-Instruct is a weaker tool user than Qwen3.5-9B at step 0 on this set (0.52 vs. 0.77) and remains 3 points behind after 100 steps, but it is still improving and its per-task solution set differs (12 tasks only the 32B policy solves). Whether it overtakes the 9B model with more steps is untested.
- **Noise.** All validation numbers are single greedy runs on 128 tasks (± 0.035 at $p = 0.8$). Peaks and single-checkpoint differences below 0.05 should not be read as real; the 80–100 means and the full curves are the reliable summaries.
- **Attribution.** Thinking statistics come from the validation outputs’ decoded text and a tag-based split; they cannot separate “useful” from “idle” thinking. The thinking-length/overlong/accuracy link in GRPO+think is a co-movement over checkpoints, not a controlled ablation.
- **Comparability with the papers** is limited to direction and gain size (Section 3.5).

References

- [1] SUPO: Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management. arXiv:2510.06727.
- [2] Generalizable End-to-End Tool-Use RL with Synthetic CodeGym. arXiv:2509.17325 (ICLR 2026).